

Sesión 12: Arquitectura y Componentes del Patrón MVC

Desacoplamiento Lógico de Software, Ciclo de Vida de las Peticiones y Enrutamiento en Aplicaciones Interactivas Modernas

IPC 2 | Facultad de Ingeniería USAC

Laboratorio - Primer Semestre 2026

| Agenda del Día (1/2)

- ✂ El Caos del Código Espagueti y la Necesidad de MVC
- 📦 El Modelo: Datos y Reglas de Estado
- 👁 La Vista: Interfaz Dinámica de Usuario
- 👤 El Controlados: Director de Orquesta y Enrutamiento

Separación de Responsabilidades

Aprenderemos a dividir aplicaciones complejas de modo que cada componente tenga una única misión perfectamente aislada.

Agenda del Día (2/2)



Ciclo de Enrutamiento

El mapa de viaje que sigue una URL web hasta transformarse en una acción ejecutable.



Sintaxis en C#

Inspección de estructuras de controladores nativas bajo ASP.NET Core.



Solidaridad

Valor de la unidad: Diseñar código limpio, legible y modular que facilite el trabajo en equipo cooperativo.

La Crisis del Código Mezclado

Por qué unificar la interfaz visual y la lógica empresarial arruina los proyectos de software

El Problema del Código Espagueti (Spaghetti Code)

En los inicios del desarrollo web, era común escribir código donde las consultas de base de datos, el procesamiento matemático y las etiquetas HTML se mezclaban en un único archivo físico.

Consecuencias del Caos: Modificar un simple color en la pantalla podía corromper accidentalmente una lógica de inserción de datos. Las pruebas unitarias se volvían imposibles de implementar y el software colapsaba bajo su propio peso de mantenimiento.

```
// Código de Conexión SQL aquí mezclado  
var datos = EjecutarQuery();  
Lista: @datos.Nombre
```


La Solución: El Patrón Arquitectónico MVC

Formulado originalmente por Trygve Reenskaug, ****Modelo-Vista-Controlador (MVC)**** es un patrón de diseño arquitectónico de software que separa la información, la interfaz de usuario y la lógica de control en tres componentes lógicos independientes.

Principio Rector: Separación de Preocupaciones (SoC)

Establece que el código debe dividirse en secciones bien delimitadas, donde cada sección aborda una responsabilidad única y aislada del sistema.

Desglose de Componentes: El Modelo

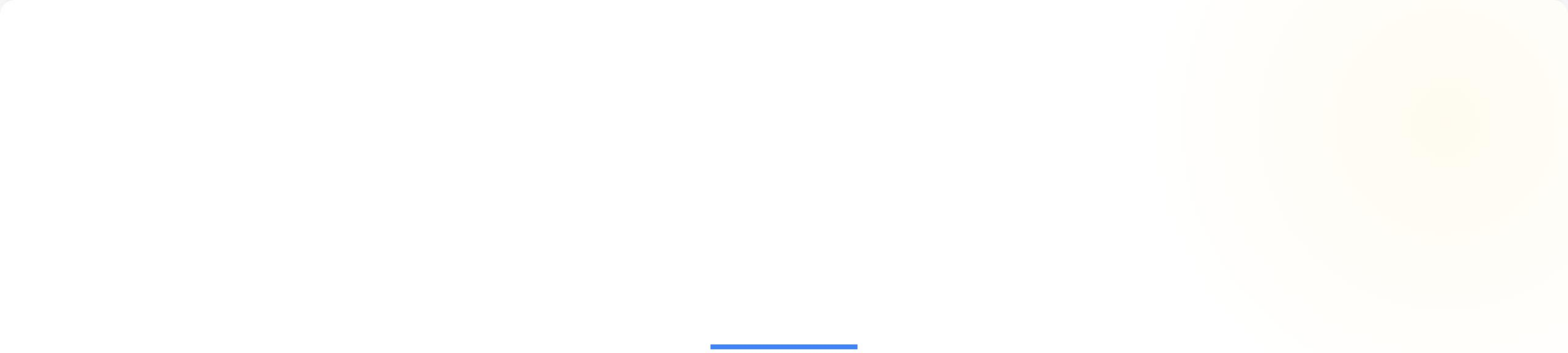
El núcleo de datos, las entidades representativas y el estado del negocio

El Modelo (Model)

Representa directamente la información y los datos reales con los que opera el sistema, además de contener las reglas y validaciones intrínsecas del negocio.

- **Modelos de Dominio:** Clases puras de C# (POCO) que mapean las propiedades de los objetos (ej. Usuario, Producto).
- **Estado:** Mantiene la coherencia de los datos en memoria antes de persistirlos.
- **Aislamiento:** No tiene conocimiento absoluto de cómo se van a mostrar los datos en pantalla o qué botón presionó el usuario.

```
public class Producto {  
    public int Id { get; set; }  
    public string Nombre { get; set; }  
    public double Precio { get; set; }  
}
```

Desglose de Componentes: La Vista

La capa de visualización encargada de transformar datos abstractos en experiencias visuales

La Vista (View)

Es la encargada de renderizar y presentar visualmente los datos contenidos en el Modelo al usuario final.

✂ Características de una Vista Limpia:

1. Es ****pasiva e inteligente****: Recibe un objeto estructurado (Modelo) y se limita a incrustar sus campos dentro de plantillas HTML.
2. ****No contiene lógica de control****: No decide si un usuario tiene permisos, no calcula impuestos complejos ni abre conexiones a bases de datos.

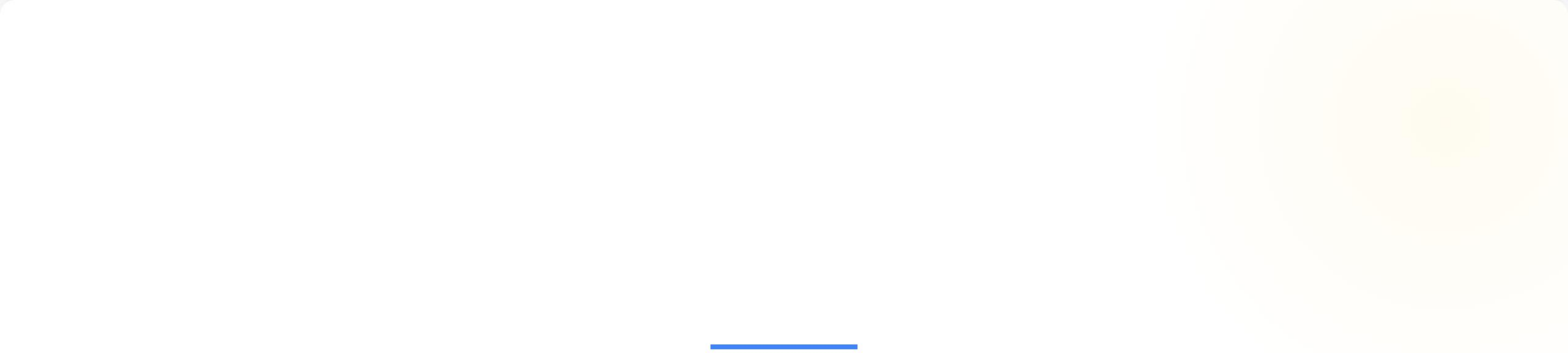
Desglose de Componentes: El Controlador

El intermediario táctico, el despachador de peticiones y gestor del flujo de ejecución

El Controlador (Controller)

Actúa como el intermediario directo entre la Vista y el Modelo. Captura las solicitudes de red del cliente, coordina qué datos buscar y decide qué vista mostrar de vuelta.

```
public class ProductoController : Controller {  
    public IActionResult Detalle(int id) {  
        Producto prod = _servicio.BuscarProducto(id); // Habla con el Modelo  
        if (prod == null) return NotFound();  
        return View(prod); // Envía el Modelo a la Vista para su renderizado  
    }  
}
```

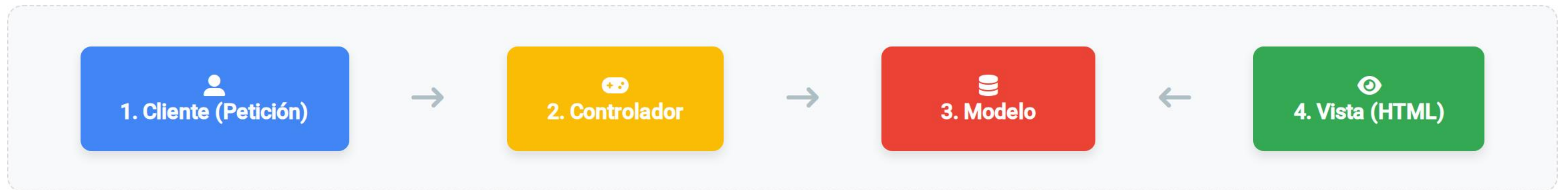



El Ciclo de Vida de una Petición MVC

Análisis paso a paso del flujo de datos interactivo desde la pulsación de un botón hasta la respuesta

Coreografía del Flujo MVC

Observemos el ciclo circular perfecto que ejecuta internamente el motor web ante cada interacción:



El cliente **nunca** interactúa directamente con el Modelo ni con la Base de Datos. El Controlador controla el acceso de forma perimetral.

Detalle del Flujo de Ejecución (Pasos 1-3)

Paso 1: Solicitud: El usuario escribe una URL o hace clic en un botón de la interfaz. Esto dispara una petición HTTP GET o POST dirigida al servidor web.

Paso 2: Intercepción del Controlador: El framework web analiza la URL entrante, crea una instancia del controlador correspondiente y delega la ejecución al método apropiado.

Paso 3: Consulta al Modelo: El controlador invoca las clases de la capa del modelo para extraer, calcular o registrar la información solicitada en la persistencia.

Detalle del Flujo de Ejecución (Pasos 4-5)

Paso 4: Enlace del Modelo a la Vista: El controlador recibe las entidades de datos limpias devueltas por el modelo y se las inyecta de forma segura a la vista correspondiente.

Paso 5: Renderizado y Entrega: La vista procesa su plantilla HTML dinámicamente sustituyendo los campos del modelo, compila el archivo final de texto plano y lo envía de regreso por la red al navegador del cliente.

Mecanismos de Enrutamiento (Routing)

Cómo mapear cadenas de texto de una URL web directamente a métodos ejecutables en clases C#

¿Qué es el Enrutamiento?

El ****Enrutamiento (Routing)**** es el motor encargado de emparejar las URLs entrantes enviadas por el cliente con un controlador y una acción de ejecución específica dentro de nuestra aplicación.

Adiós a las extensiones físicas: En arquitecturas obsoletas, las URLs hacían referencia directa a archivos del disco duro (ej. /tienda/productos.aspx). Con el enrutamiento de MVC, las URLs se vuelven ****semánticas y limpias**** (ej. /productos/detalle/5).

Enrutamiento Convencional en .NET

Por convención estándar, el motor de ASP.NET Core viene pre-configurado con una plantilla de enrutamiento por defecto estructurada en tres segmentos jerárquicos:

```
"{controller=Home}/{action=Index}/{id?}"
```

{controller}

Determina a qué clase controladora se enviará la petición.

{action}

Define el nombre del método concreto que se ejecutará dentro de la clase.

{id?}

Parámetro opcional para identificar un registro único de datos.

Ejemplo Práctico de Mapeo de URL

Analicemos matemáticamente cómo el framework traduce en tiempo de ejecución la siguiente ruta de red:

```
https://misitio.com/Estudiante/Historial/20260123
```

- El motor busca una clase llamada: EstudianteController
- Dentro de esa clase, manda a ejecutar el método: `public IActionResult Historial(int id)`
- Inyecta de forma automática en el argumento `id` el valor entero: 20260123

Beneficios de la Arquitectura MVC en Ingeniería

Análisis de métricas de calidad de software asociadas a patrones de desacoplamiento lógico

Alta Cohesión y Bajo Acoplamiento

MVC es la herramienta por excelencia para cumplir con dos de las métricas de diseño de software arquitectónico más importantes de la ingeniería:

1. Alta Cohesión

Cada componente se enfoca estrictamente en su naturaleza. Las vistas solo saben renderizar HTML; los modelos solo saben validar datos del negocio. Cada clase hace una sola cosa bien.

2. Bajo Acoplamiento

Las dependencias inter-clases se reducen al mínimo formal. Cambios estructurales profundos en el diseño visual de la interfaz no rompen ni obligan a compilar de nuevo las clases de acceso a datos.

Desarrollo en Paralelo y Trabajo Eficiente

En ambientes corporativos con equipos numerosos, la mezcla de código genera bloqueos y conflictos de fusión de ramas (Git Merge Conflicts) masivos.

Flujo de Trabajo Colaborativo Puro: Gracias al aislamiento de MVC, un programador frontend puede refactorizar y embellecer por completo los archivos de las ****Vistas****, mientras simultáneamente un programador backend reescribe los algoritmos de optimización del ****Modelo****, todo sin pisarse el código ni generar bloqueos de archivos.

Buenas Prácticas de Ingeniería en MVC

Patrones de desarrollo para evitar la degradación arquitectónica en aplicaciones web corporativas

Antipatrón: Fat Controllers (Controladores Gordos)

El error conceptual más grave cometido en el desarrollo de MVC es escribir código de negocio, validaciones tributarias o consultas de SQL directo dentro de los métodos del controlador.

Regla de Oro de la Industria (Skinny Controllers): Los controladores deben ser delgados. Su única función es ****redirigir el tráfico****. Si un método de controlador supera las 15 o 20 líneas de código, significa que estás programando lógica empresarial en el lugar equivocado; delega esa responsabilidad al Modelo.

Dimensión Ética del Software Arquitectónico

La relación directa entre la limpieza del código fuente y nuestra responsabilidad con los demás

Valor de la Unidad: Solidaridad



"Llevamos dentro un impulso de solidaridad profunda que nos vincula a la suerte de los demás."

— **Helen Keller**

****Aplicación en el Desarrollo de Software:**** Escribir código ordenado siguiendo el patrón MVC es una demostración directa de ****solidaridad con tus compañeros ingenieros****. Un código limpio, documentado y desacoplado previene el estrés laboral, facilita la integración de nuevos desarrolladores al equipo y asegura que el sistema pueda crecer colectivamente sin volverse una pesadilla para quienes hereden tu trabajo en el mañana.

Conclusiones de la Sesión

Tríada Lógica

MVC segmenta de forma estricta las tareas de persistencia de datos (M), la representación visual pasiva (V) y el enrutamiento de peticiones (C).

Rutas Semánticas

El enrutamiento moderno independiza las URLs lógicas de la ubicación de los archivos físicos en el disco del servidor.

Mantenibilidad

Alcanzar una alta cohesión lógica y bajo acoplamiento físico disminuye exponencialmente los costos globales de mantenimiento del software.

Próximo Paso: Sesión 13

Mapeo y Enlace de Datos mediante Protocolo HTTP

En la siguiente clase aprenderemos a enviar y capturar información compleja entre capas. Estudiaremos a fondo la ****Sesión 13: Binding de datos, transferencia de parámetros y estructuración de peticiones HTTP****, uniendo vistas y controladores de forma dinámica.

¿Dudas sobre los Componentes o el Flujo del Patrón MVC?

Abramos el espacio para comprender a fondo la arquitectura lógica de nuestro código.



Foros de Discusión en UEDI



Canal de Soporte: 3021894750101@ingenieria.usac.edu.gt